CHAPTER 2

The SNMP Crawler

Walking the hallways, knocking on every room

What this chapter is about

The firewall told us which subnets exist and how to reach them. Now we walk into the building — into the switches and routers themselves — and ask each one three questions: *"who are you?"*, *"who are your neighbours?"*, and *"who's plugged into your ports right now?"*. We do this recursively, neighbour to neighbour, until we've mapped the whole network. The output is a **topology graph**: every switch, every uplink between switches, every host with the exact port it's plugged into.

> ***Real-life analogy.*** *Imagine you've just been handed a list of floors in a 30-storey building. Now you take a clipboard and walk into floor 3. There's a panel on the wall called the wiring closet. You read it: "I am Switch-3A, my neighbour through cable 7 is Switch-3B, my neighbour through cable 12 is Switch-2-Core." You write that down. Then you walk to Switch-3B. You read its panel. It points to Switch-3A (you've been there) and to Switch-3C (new — go there next). You also peek at the desk-occupancy log on each switch ("desk #14 is currently used by Mr. Patel's laptop") and you write that down too. By the time you finish, you have the full wiring diagram of the building plus a list of who is sitting at which desk. The clipboard-and-walking is the SNMP Crawler.*

Words you'll see (and what they mean)

> **SNMP (Simple Network Management Protocol).** A very old, very widely supported protocol that lets you read structured information out of a network device. Think of it as "the device exposes a giant tree of facts; you can read any branch of the tree." Versions in the wild: SNMPv1 (avoid — plaintext, no auth), SNMPv2c (still common — uses a "community string" as a shared password), SNMPv3 (modern — username + authentication + encryption).

> **Community string.** The "password" for SNMPv2c. Conventionally `public` for read-only. Every customer should have changed this; many haven't.

OID (Object IDentifier). A long dotted number that names one branch of the SNMP tree, like `1.3.6.1.2.1.2.2.1.2`. Each OID has a human-readable name in a "MIB" (see below).

MIB (Management Information Base). A documentation file that tells you "this OID is called `ifDescr` and it holds the human-readable name of an interface, like Gi1/0/1." MIBs are like the Yellow Pages for OIDs.

SNMP walk. "Give me everything under this branch of the tree." If the branch is "interface table", you get back one row per interface on the device.

GETBULK. A bulk-read operation in SNMPv2c+. We tell the device "give me 50 OIDs at a time" instead of 1, which makes walks 10–50× faster.

LLDP (Link Layer Discovery Protocol). An IEEE standard. Every modern switch shouts "I am Switch-A on port Gi1/0/3" out of every port every 30 seconds. Its neighbour hears this and stores it in a table. We read that table to learn the wiring. Vendor-neutral.

CDP (Cisco Discovery Protocol). Same idea as LLDP, but Cisco-only and older. Many Cisco shops disable LLDP for "security" and rely on CDP. So we always read both.

Layer 2 (L2) and Layer 3 (L3). Layer 2 = ethernet — switches forwarding frames using MAC addresses. Layer 3 = IP — routers forwarding packets using IP addresses. A switch is L2; a router is L3; an "L3 switch" does both.

VLAN (Virtual LAN). A way to slice one physical switch into many isolated logical networks. Cable-wise the ports are all on one box; logically the "Sales VLAN" and the "HR VLAN" can't talk to each other without going through a router.

Routing table / RIB (Routing Information Base). Same idea as on the firewall — "to reach this network, send packets to that next-hop." On routers we read it to learn what other networks exist beyond what the firewall already told us.

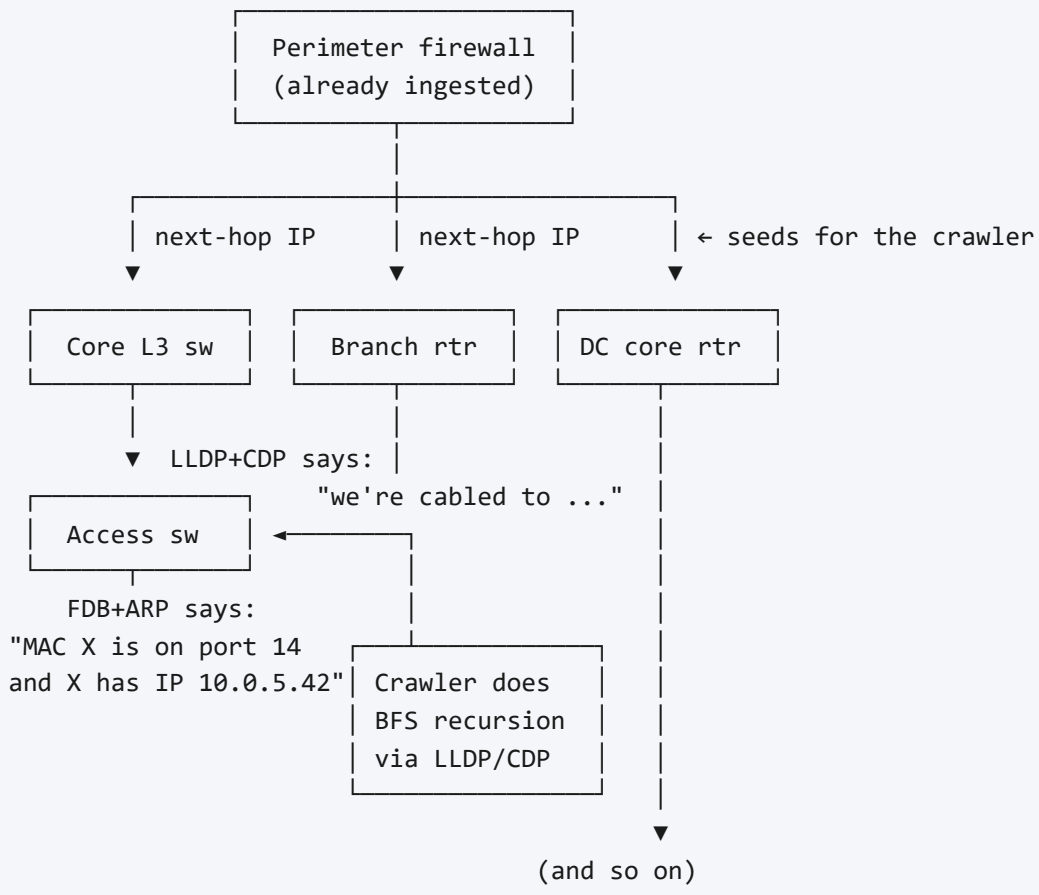
ARP table / IP-to-MAC binding. "I just spoke to IP 10.0.5.42 and its MAC is aa:bb:cc:..." On switches and routers, walking this gives us a list of currently-active hosts. The modern OID branch is called `ipNetToPhysicalTable`; older devices only populate the legacy `ipNetToMediaTable`, so we walk both.

MAC table / FDB (Forwarding DataBase) / CAM (Content-Addressable Memory) table. Same thing, three names. The switch's record of "MAC aa:bb:cc:... was last seen on port Gi1/0/14." Joining the FDB (MAC→port) with the ARP table (IP→MAC) lets us say "host 10.0.5.42 is plugged into port 14 of switch-3B." That join is the Holy Grail of host placement.

BFS (Breadth-First Search). The simplest graph-walking algorithm: "look at all my neighbours first, then their neighbours, then theirs." Compare to depth-first ("go deep down one path before backing up"). BFS is the right algorithm for topology because it discovers the whole "first ring" of switches before going further out.

Chassis ID. A unique serial number that identifies one physical box. We use it as a deduplication key — if a switch has 3 management IPs, we don't want to walk it 3 times. Chassis ID says "you've already met this box."

What we're building (the picture)



The output we hand to Chapter 4 is a graph: **nodes** are devices, **edges** are LLDP/CDP cables between switches plus ARP/FDB joins for host placement.

What we read on each device, and why

MIB table (with OID)	What it gives us
System group — `sysDescr.0`, `sysObjectID.0`, `sysName.0` (1.3.6.1.2.1.1)	"I am a Cisco Catalyst 9300, software 17.6, hostname swA-floor3." Always read first.
IF-MIB::ifTable + ifXTable (1.3.6.1.2.1.2.2 and .31.1.1)	List of every interface: name, type, speed, up/down, MAC. Use `ifHighSpeed` for accurate speeds (the older `ifSpeed` wraps at 4 Gbps).
LLDP-MIB::lldpRemTable (1.0.8802.1.1.2.1.4.1.1)	"On my port 5 my neighbour is sysName=swB, port=Gi1/0/3, mgmt-IP=10.0.0.2." This is what drives recursion.
CISCO-CDP-MIB::cdpCacheTable (1.3.6.1.4.1.9.9.23.1.2.1.1)	Same as LLDP, Cisco flavour. Walk in parallel — many shops have one or the other disabled.
IP-MIB::ipNetToPhysicalTable (1.3.6.1.2.1.4.35), fallback `ipNetToMediaTable` (.4.22)	ARP table: which IP is currently bound to which MAC.
BRIDGE-MIB::dot1dTpFdbTable (1.3.6.1.2.1.17.4.3) or **Q-BRIDGE-MIB::dot1qTpFdbTable**	Switch's MAC-to-port table. Joined with ARP, gives us host placement.
IP-FORWARD-MIB::inetCidrRouteTable (1.3.6.1.2.1.4.24.7), fallback `ipCidrRouteTable` (.24.4)	Routing table on L3 devices.
IP-MIB::ipAddressTable (1.3.6.1.2.1.4.34)	Which IPs are configured on which interfaces. Two routers on the same /30 subnet → L3 adjacency edge.
ENTITY-MIB::entPhysicalTable (1.3.6.1.2.1.47.1.1.1.1)	Chassis serial number, model, modules. Use the chassis-class entry's `entPhysicalSerialNum` as your dedup key.

The recursion algorithm in plain English

This is the heart of the crawler. We use **BFS**:

```
queue   = [ seeds from firewall RIB ]    # IPs of next-hop routers
visited = { }                            # set of chassis IDs we've already crawled

while queue is not empty:
    target = queue.pop_front()

    # Step 1: do we have credentials that match this IP/range?
    creds = credentialStore.match(target.ip, tenant)
    if creds is None:
        graph.markUnreached(target, reason = "no credentials")
        continue

    # Step 2: try to talk SNMP
    conn, err = snmp.dial(target.ip, creds, timeout=2s, retries=2)
    if err:
        graph.markUnreached(target, reason = err)
        continue

    # Step 3: figure out who this device IS — chassis ID for dedup
    chassisID = walk(entPhysicalSerialNum, class=chassis)
                or walk(lldpLocChassisId)
    if chassisID in visited:
        graph.aliasIP(target.ip, chassisID)   # different IP, same box
        continue
    visited.add(chassisID)

    # Step 4: read the device
    device = collect(sysInfo, ifTable + ifXTable, ipAddressTable)
    graph.addNode(device)

    # Step 5: find neighbours
    neighbours = walk(lldpRemTable) + walk(cdpCacheTable)
    for n in neighbours:
        graph.addEdge(device.localPort, n)
        if n.chassisId not in visited and n.chassisId not in queue:
            queue.push({chassisID: n.chassisId, candidateIPs: n.mgmtAddrs})

    # Step 6: enrich (does NOT drive further recursion)
    fdb    = walkFDB(device, perVLAN = isCisco(device))
    arp    = walk(ipNetToPhysical) or walk(ipNetToMedia)
    routes = walk(inetCidrRoute) or walk(ipCidrRoute)
    graph.attachHosts( join(fdb, arp) )
```

Two crucial design choices:

1. **Dedup by chassis ID, not by IP.** Routers often have many management IPs (HSRP/VRRP virtual IPs, multiple loopbacks, stacked switches sharing one chassis). If you dedup by IP you'll walk the same box six times.

2. **Cap the BFS.** A misconfigured network can return millions of fake neighbours. Cap depth (e.g. 8 hops) and total node count per tenant (e.g. 10,000 devices) as a circuit breaker.

The Cisco VLAN-context trick (don't skip this)

On Cisco IOS / IOS-XE switches, the **FDB / MAC-table walk** is broken in a famous way: a plain SNMPv2c walk only returns MACs from the *native VLAN*. To see MACs in VLAN 10, you must connect with community string `public@10`. To see VLAN 20, use `public@20`. And so on.

SNMPv3 has a clean equivalent: set the `contextName` field to `vlan-10`.

So the algorithm becomes:

1. Walk `vtpVlanState` (or just `1.3.6.1.2.1.17.7.1.4.3.1.4`) to enumerate the active VLANs.
2. For each VLAN, re-connect with the appropriate context and walk the FDB.
3. Union all results.

NX-OS and most non-Cisco vendors don't need this; they expose VLAN-aware data through Q-BRIDGE-MIB directly.

Performance and politeness

Real-world numbers on a 48-port access switch with 50 ms RTT:

- System group + ifTable + ifXTable walk: 0.5 to 1.5 seconds.
- FDB walk for one VLAN: 0.5 to 2 seconds. With 50 VLANs sequentially: 30–90 seconds. *This dominates total walk time on Cisco.*
- LLDP + CDP walk: under a second combined.
- Routing table on a core router with 50,000 routes: 20–60 seconds. Diff-walk if you can.

Politeness rules:

1. **One SNMP session per device, walks done sequentially.** Many low-end switches drop packets above one outstanding request and have CPU-throttled SNMP that caps at ~50 packets/sec. A misbehaving collector can spike a switch's CPU and page the customer's network team at 3am.
2. **Across devices, parallel.** A worker pool of ~32 max, scaled per tenant.
3. **Stagger walk start times.** Don't fire the entire fleet at `:00` on the hour.
4. **Backoff.** Exponential on timeout; blacklist after 3 consecutive failures within a poll cycle.

5. **Distinguish failures.** "Auth failed" (bad community/v3 creds) is different from "unreachable" (no UDP response). Operators need to see which.

How we test Chapter 2

Lab setup. Build a small known topology in EVE-NG or GNS3 (free emulator):

```
coreR  ── distSw ── accessSw1 ── pcA, printerB
                  └ accessSw2 ── pcC, vmHostD
```

Configure SNMPv2c read-only with a known community on every device. Optionally enable SNMPv3 on one device to test the v3 path.

Test 1 — Identity and interfaces.

1. Run the crawler with `coreR` as the seed.
2. Confirm every device's `sysName` appears in `device` table.
3. Confirm interface counts match what you configured.

Test 2 — Edges from LLDP/CDP.

1. The graph must contain edges `coreR↔distSw`, `distSw↔accessSw1`, `distSw↔accessSw2`.
2. Disable LLDP on one device, leave CDP on; rerun. Edges still appear.
3. Disable both; that device is reachable but its neighbours edge from *only* the other side. Confirmed expected behaviour.

Test 3 — Host placement.

1. Confirm `pcA`'s IP is associated with `accessSw1` port X — joining FDB and ARP correctly.
2. Confirm a host that has been silent for > ARP-timeout no longer shows up (shows in observation history but not "current state").

Test 4 — VLAN context trick on Cisco.

1. Configure VLAN 10 and VLAN 20 on accessSw1 with hosts in each.
2. Without the trick, plain walk only sees VLAN 1's MACs. With the trick, see all.

Test 5 — Chassis-ID dedup.

1. Configure two management IPs on coreR.
2. Seed the crawler with both.
3. Confirm only one `device` row exists, with both IPs aliased.

Test 6 — Politeness under load.

1. Build a 100-device emulated lab.
2. Run a full crawl. Monitor CPU on the smallest emulated switch.
3. Confirm CPU stays under 50% during the walk.

Things that can go wrong (and what to watch for)

LLDP disabled on user ports as a "security policy". You will see uplinks but no edges to APs, IP phones, or end-host bridges. This is normal in security-conscious enterprises; don't try to "fix" it. Document the gap.

Stale ARP/FDB entries. A host that pinged 4 hours ago still appears. Always store the `seen_at`; treat anything older than 1 ARP-aging-cycle (default 4 hours on most gear) as "last seen", not "currently here."

Phones daisy-chaining laptops. An IP phone with a built-in switch shows up as one LLDP-MED neighbour, plus several MACs behind that single port via FDB. The laptop is one of those MACs, not a port-direct neighbour. Your fusion logic must understand "MAC behind port X" vs "device cabled to port X" are different.

Unmanaged switches. A cheap unmanaged 8-port switch is invisible to SNMP. Its uplink port on the upstream device shows 8 MACs, all "behind that port." You can't distinguish which is plugged where. Document the gap; flag it as "MAC cloud".

Cisco SMB / MikroTik fragility. These rate-limit SNMP harshly. Treat them with extra politeness: max 50 GETBULK/sec, 2-second inter-PDU delay if flagged "fragile."

SNMPv3 EngineID discovery latency. The first packet to a v3 device costs an extra round-trip. Reuse the SNMP session struct across walks to one device, don't rebuild it per

OID.

Wireless clients invisible from switch alone. WiFi clients show up as a sea of MACs behind the AP's switch port. To tell them apart you need the wireless controller's MIB (CISCO-LWAPP-MIB, ARUBA-WLAN-MIB) — defer this to a future phase.

Exit checklist (you can move to Chapter 3 only when ALL are true)

- ☐ A 100-device emulated topology produces a graph that matches the cabling diagram.
- ☐ LLDP-only and CDP-only and both-enabled cases all yield correct edges.
- ☐ Cisco per-VLAN community trick is implemented and tested with at least 2 VLANs.
- ☐ Chassis-ID dedup proven with a 2-mgmt-IP device.
- ☐ Auth-failure and unreachable are distinguished in results and audit log.
- ☐ A full crawl on the lab leaves all device CPUs under 50%.
- ☐ ARP+FDB join correctly places at least 90% of test hosts onto the right port.
- ☐ BFS depth and node-count caps are configurable and tested with a runaway scenario.

Recap card — Chapter 2 in 5 sentences

1. Seeded from the firewall's next-hop IPs, the crawler walks SNMP on every router/switch and recursively follows LLDP+CDP neighbour pointers.
2. On every device read system info, interfaces (IF-MIB), neighbours (LLDP-MIB + CDP-MIB), ARP (IP-MIB), MAC table (Q-BRIDGE-MIB), and routes (IP-FORWARD-MIB).
3. Use BFS, dedup by chassis serial (not IP), cap depth and node-count to prevent runaway walks.
4. Cisco IOS needs the per-VLAN community trick (`community@vlan-id`) for FDB; without it you only see one VLAN of MACs.
5. Be polite — sequential walks per device, parallel across devices, capped concurrency, exponential backoff, distinguish auth-failure from unreachable.